



FLOWDIARY

AI-Powered Ethical Hacking

Lecture 8 — HTTP Requests & Responses in Depth

Anatomy of a request & response · methods · headers · query strings vs body · content types · HTTPS / TLS basics

Tutor: Engr. Ibrahim Auwal

Why this lecture matters for ethical hackers

Lectures 6 and 7 showed you the network journey and the web technologies at each end. HTTP is the actual language spoken between them — and it is the single most important protocol you will manipulate as an ethical hacker.

Almost every web attack is a crafted HTTP message. Injecting a payload, forging a header, tampering with a cookie, brute-forcing a login — all of it is just modified requests. Tools like Burp Suite exist to let you read and rewrite these messages. To use them well, you must know exactly what a request and response are made of.

Learning Objectives

By the end of this lecture you should be able to:

- Identify every part of an HTTP request and an HTTP response.
- Explain the common HTTP methods and when each is used.
- Describe what headers do and name the security-relevant ones.
- Distinguish query-string data from body data and know when each is used.
- Explain content types and why they matter.
- Explain what HTTPS and TLS add, and what they do and do not protect.

1. The Anatomy of an HTTP Request

Every time a browser or tool talks to a web server, it sends a request: a plain-text message with a strict structure. Once you can read this structure, the web stops being magic. A request has four parts, in order.

A complete HTTP request

```

POST /login HTTP/1.1          ← 1. Request line
Host: example.com             ← 2. Headers
Content-Type: application/json
Content-Length: 44
Cookie: session=abc123

                                ← 3. Blank line
{ "username": "ibrahim",      ← 4. Body
  "password": "secret" }

```

1.1 The request line

The first line states three things: the method (what action to perform, e.g. POST), the path (which resource, e.g. /login), and the HTTP version. This single line tells the server the intent of the whole message.

1.2 The headers

Headers are key-value lines that carry metadata about the request: which host to reach, what kind of data is enclosed, what cookies the client holds, what browser is being used, and more. They describe and shape the request without being the content itself.

1.3 The blank line and body

A single empty line separates the headers from the body. The body carries the actual payload — for example the login credentials, a submitted form, or uploaded data. Not every request has a body: a simple GET usually has none.

Security lens: the request is fully attacker-controlled

Everything in a request — the path, every header, the cookies, and the body — is sent by the client and can be changed by the client. A browser sends normal values; an attacker using an intercepting proxy can send anything.

Core principle: never trust any part of an incoming request. The server must validate all of it, because none of it is guaranteed to be honest.

2. The Anatomy of an HTTP Response

The server replies with a response, which mirrors the request's structure. It too has a status line, headers, a blank line, and usually a body.

A complete HTTP response

```

HTTP/1.1 200 OK              ← 1. Status line
Content-Type: text/html       ← 2. Headers
Content-Length: 1256
Set-Cookie: session=abc123; HttpOnly

                                ← 3. Blank line
<!DOCTYPE html>              ← 4. Body
<html> ... </html>

```

2.1 The status line and status codes

The first line reports how the request went, using a three-digit status code. The first digit groups the outcome. You met several of these in Lecture 6; here is the full family.

Range	Category	Meaning
1xx	Informational	Request received, continuing.
2xx	Success	It worked (e.g. 200 OK, 201 Created).
3xx	Redirection	Look elsewhere (e.g. 301, 302).
4xx	Client error	Your request was wrong (400, 401, 403, 404).
5xx	Server error	The server failed (500, 502, 503).

2.2 Response headers and body

Response headers describe what is being returned: the content type, its length, caching rules, and any cookies the server wants to set. The body is the actual content — an HTML page, a JSON payload, an image, or a file. The client uses the headers to decide how to handle the body.

Security lens: responses leak information

Server / X-Powered-By headers can reveal exact software and versions — instant fingerprinting for an attacker (recall Lecture 7).

Verbose error bodies (5xx) may expose stack traces, file paths, or database details.

Missing security headers in the response is itself a finding — it tells you defenses are absent.

3. HTTP Methods

The method is the verb of the request — it says what you want to do to the resource. You saw these briefly with REST APIs in Lecture 7; here we look at them as raw HTTP.

Method	Purpose	Has body?
GET	Retrieve a resource. Should not change data.	No
POST	Submit data / create something.	Yes
PUT	Replace a resource entirely.	Yes
PATCH	Partially update a resource.	Yes
DELETE	Remove a resource.	Sometimes
HEAD	Like GET but headers only, no body.	No
OPTIONS	Ask which methods the server allows.	No

3.1 Safe and idempotent methods

Two properties matter. A safe method (like GET) is not supposed to change anything on the server. An idempotent method produces the same result no matter how many times you repeat it (GET, PUT, and DELETE are idempotent; POST is not, since repeating it may create duplicates). These are conventions the server should honor — not guarantees enforced by HTTP itself.

Security lens: methods are testable and abusable

GET should never change data. If an app deletes or transfers something via GET, that action can be triggered by a hidden link or image — the basis of CSRF attacks.

OPTIONS and unusual methods can reveal functionality the UI hides. Testers probe which methods an endpoint actually accepts.

Method tampering: some access controls check only one method; switching GET to POST (or vice versa) can bypass weak checks.

4. Headers in Depth

Headers do most of the quiet work in HTTP. There are many, but for security work a handful matter most. They fall into request headers, response headers, and security headers.

4.1 Commonly seen headers

Header	Direction	What it does
Host	Request	Which site on the server you want.
User-Agent	Request	Identifies the browser / client.
Cookie	Request	Sends stored session data back.
Authorization	Request	Carries credentials / tokens.
Set-Cookie	Response	Tells the browser to store a cookie.
Content-Type	Both	The format of the body.
Location	Response	Where to redirect to.

4.2 Security headers worth knowing

Some response headers exist purely to harden the browser's behavior. Their presence (or absence) is a routine thing to check during an assessment.

- **Strict-Transport-Security (HSTS):** forces the browser to use HTTPS only.
- **Content-Security-Policy (CSP):** restricts what scripts can run — a strong defense against XSS.

- **X-Frame-Options:** blocks the page from being framed, preventing clickjacking.
- **HttpOnly / Secure (on cookies):** hide cookies from JavaScript and restrict them to HTTPS.

Security lens: cookies are a prime target

Cookies often hold the session token that proves who you are. If an attacker steals it, they become you — no password needed. That is why the HttpOnly and Secure flags matter: HttpOnly blocks JavaScript (and therefore XSS) from reading the cookie, and Secure stops it being sent over plain HTTP.

Testers routinely check: are session cookies flagged correctly, and can the token be predicted, replayed, or fixed?

5. Query Strings vs Body — Where Data Goes

When a client sends data, it can travel in two main places: appended to the URL as a query string, or inside the request body. Knowing the difference is essential for both testing and understanding what gets logged.

5.1 The query string

A query string is attached to the end of the URL after a question mark, as key-value pairs joined by ampersands. For example:

```
https://example.com/search?q=firewall&page=2
```

Here `q=firewall` and `page=2` are the data. Query strings are typically used with GET requests, for things like search terms, filters, and page numbers.

5.2 The request body

The body carries data inside the message rather than in the URL. It is used with POST, PUT, and PATCH for larger or more sensitive payloads — login credentials, form submissions, JSON documents, or file uploads.

	Query string	Request body
Location	In the URL	In the message body
Typical method	GET	POST / PUT / PATCH
Visible in URL?	Yes	No
Logged / cached / bookmarked?	Often yes	Usually no
Good for secrets?	No — leaks easily	Better, but still needs HTTPS

Security lens: why this distinction matters

Never put secrets in a query string. URLs are logged by servers and proxies, stored in browser history, and leaked via the Referer header. A password or token in the URL is a password or token in a dozen log files.

Both are injection points. Query parameters and body fields are both attacker-controlled inputs — prime places to test for SQL injection, XSS, and parameter tampering.

6. Content Types

The Content-Type header tells the receiver how to interpret the body. The same bytes could be a web page, a data payload, or an image — the content type removes the ambiguity. It is also called the MIME type.

Content-Type	Meaning	Typical use
text/html	An HTML document	Web pages
application/json	JSON data	API requests & responses
application/x-www-form-urlencoded	Encoded form fields	Classic HTML form POSTs
multipart/form-data	Form data + files	File uploads
text/plain	Plain text	Simple text bodies
image/png, image/jpeg	Image data	Images

Security lens: content types get abused

Content-Type confusion: changing the declared type can bypass filters or trick a parser — e.g. sending JSON where a form is expected, or vice versa, to slip past validation.

Upload bypasses: with multipart/form-data uploads, attackers manipulate the declared type or filename to sneak in malicious files.

MIME sniffing: if a server sends the wrong type, a browser may guess and execute content as script — a classic XSS vector.

7. HTTPS and TLS Basics

Everything so far describes plain HTTP, which travels in readable text. Anyone positioned on the network — the man-in-the-middle from Lecture 6 — could read or alter it. HTTPS fixes this by wrapping HTTP inside TLS (Transport Layer Security). The "S" means secure.

7.1 What TLS provides

TLS adds three guarantees to the conversation:

- **Confidentiality:** the traffic is encrypted, so eavesdroppers see only scrambled data.
- **Integrity:** tampering is detectable, so data cannot be silently altered in transit.
- **Authentication:** a certificate proves you are really talking to the site you intended, not an impostor.

7.2 How the TLS handshake works (simplified)

Before any HTTP is exchanged, the client and server perform a TLS handshake — the step you saw named in Lecture 6's journey:

1. **Client hello.** The browser proposes TLS versions and cipher options.
2. **Certificate.** The server presents its certificate, proving its identity via a trusted Certificate Authority.
3. **Key exchange.** Both sides securely agree on a shared secret key without ever sending it in the clear.
4. **Encrypted channel.** From here on, all HTTP traffic is encrypted with that key.

Security lens: what TLS does and does NOT protect

It protects data in transit. Someone sniffing the network cannot read or change the traffic.

It does NOT make the app safe. SQL injection, XSS, broken auth, and every other flaw work perfectly well over HTTPS. The lock icon means "the tunnel is private," not "the site is secure."

It does NOT protect the endpoints. Once traffic arrives, the server (or a compromised browser) sees it in plain form. Intercepting proxies like Burp read HTTPS precisely because they sit at an endpoint.

Watch for: expired or self-signed certificates, weak/old TLS versions, and HTTP pages that never redirect to HTTPS.

8. Recap and Key Takeaways

- **A request has four parts** — request line, headers, blank line, body — and every part is attacker-controlled.
- **A response mirrors it** with a status line, headers, and body, and often leaks useful information.
- **Methods are verbs;** GET should never change data, and method behavior is routinely tested and abused.
- **Headers carry the metadata** — including cookies and security headers whose presence or absence is a finding.
- **Query string vs body** decides visibility and logging; never put secrets in the URL.
- **Content types tell the receiver how to read the body** and are a real source of bypasses.

- **HTTPS/TLS secures the tunnel, not the app** — it protects data in transit and nothing more.

Discussion & Lab Prompts

1. Use browser dev tools (Network tab) or `curl -v` to capture a real request and response. Label every part against Sections 1 and 2.
2. Submit a form and inspect whether the data went in the query string or the body. Why did the developer choose that?
3. Inspect the response headers of three sites. Which security headers are present? Which are missing?
4. Check a site's certificate details. What TLS version and issuer does it use? Does HTTP redirect to HTTPS?
5. Discuss: a site uses HTTPS but is still hacked via SQL injection. Explain to a beginner why the padlock did not help.

Key Terms Glossary

Term	Quick definition
Request line	First line of a request: method, path, version.
Status code	Three-digit result code in a response.
Header	Key-value metadata line in a message.
Body	The payload/content of a message.
Method	The action verb (GET, POST, ...).
Idempotent	Repeating it gives the same result.
Query string	Data in the URL after '?'.
Content-Type	Declares the format of the body (MIME type).
Cookie	Stored data sent back with each request.
TLS / HTTPS	Encryption layer securing HTTP in transit.